

Ayudantía: Estructuras de Datos y Algoritmos (EDA)

Material de Apoyo Práctico

Índice

1. Funciones Recursivas	2
1.1. Explicación Técnica	2
1.2. Diagrama de Árbol de Llamadas	2
1.3. Ejemplo y Complejidad	2
1.4. Ejercicio de Práctica en LeetCode	2
2. Memoización	3
2.1. Explicación Técnica	3
2.2. Diagrama de Flujo con Caché	3
2.3. Ejemplo y Complejidad	3
2.4. Ejercicio de Práctica en LeetCode	4
3. Dividir y Conquistar (Divide and Conquer)	5
3.1. Explicación Técnica	5
3.2. Diagrama de Componentes del Algoritmo	5
3.3. Ejemplo y Complejidad	5
3.4. Ejercicio de Práctica en LeetCode	5
4. Merge Sort	6
4.1. Explicación Técnica	6
4.2. Diagrama de Fase de División	6
4.3. Ejemplo y Complejidad	6
4.4. Ejercicio de Práctica en LeetCode	6
5. Búsqueda Binaria (Binary Search)	7
5.1. Explicación Técnica	7
5.2. Diagrama de Punteros	7
5.3. Ejemplo y Complejidad	7
5.4. Ejercicio de Práctica en LeetCode	7
6. Listas Enlazadas (Linked Lists)	8
6.1. Explicación Técnica	8
6.2. Diagrama de Clase y Objetos	8
6.3. Ejemplo y Complejidad	8
6.4. Ejercicio de Práctica en LeetCode	8

1. Funciones Recursivas

1.1. Explicación Técnica

Una función recursiva es aquella que se llama a sí misma durante su ejecución. Todo algoritmo recursivo debe tener al menos dos partes fundamentales:

- **Caso base:** La condición de parada que evita que la función se llame infinitamente.
- **Paso recursivo:** La lógica que reduce el problema original a un subproblema más pequeño.

1.2. Diagrama de Árbol de Llamadas

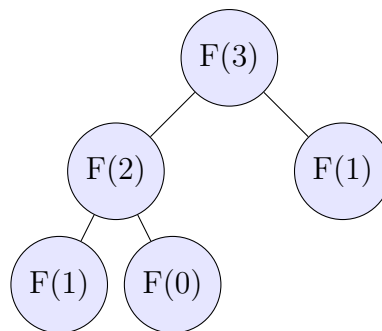


Diagrama: Representación del estado de las llamadas recursivas (ej. cálculo de Fibonacci).

1.3. Ejemplo y Complejidad

Ejemplo clásico: Cálculo del factorial de un número N .

```
1 public int factorial(int n) {  
2     if (n == 0) return 1; // Caso base  
3     return n * factorial(n - 1); // Paso recursivo  
4 }
```

Complejidad Temporal: $\mathcal{O}(N)$, ya que la función se llama N veces en profundidad.

Complejidad Espacial: $\mathcal{O}(N)$, debido a la memoria requerida por la pila de llamadas (Call Stack).

1.4. Ejercicio de Práctica en LeetCode

Nombre: Fibonacci Number

Número: 509

Dificultad: Fácil

Link: <https://leetcode.com/problems/fibonacci-number/>

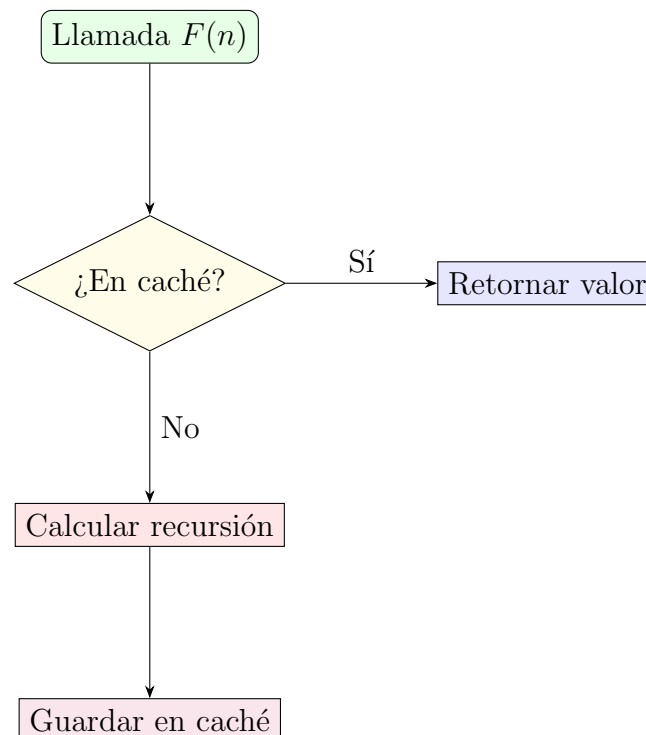
Complejidad Esperada: $\mathcal{O}(2^N)$ (aproximación ingenua por recursión múltiple sin optimizar).

2. Memoización

2.1. Explicación Técnica

La memoización es una técnica de optimización (parte de la Programación Dinámica en su enfoque *Top-Down*) que consiste en almacenar los resultados de las llamadas a funciones costosas y devolver el resultado en caché cuando ocurren los mismos inputs nuevamente. Se usa mucho junto a la recursividad para evitar la explosión combinatoria de subproblemas repetidos.

2.2. Diagrama de Flujo con Caché



2.3. Ejemplo y Complejidad

Ejemplo: Fibonacci optimizado usando un arreglo para almacenar estados ya calculados.

```
1 int[] memo = new int[N + 1];
2 public int fib(int n) {
3     if (n <= 1) return n;
4     if (memo[n] != 0) return memo[n]; // Consulta al caché
5     memo[n] = fib(n - 1) + fib(n - 2); // Guardado en caché
6     return memo[n];
7 }
```

Complejidad Temporal: $\mathcal{O}(N)$, ya que cada subproblema único (del 0 al N) se calcula estrictamente una vez.

Complejidad Espacial: $\mathcal{O}(N)$ para el arreglo de memoización más la pila de llamadas.

2.4. Ejercicio de Práctica en LeetCode

Nombre: Climbing Stairs

Número: 70

Dificultad: Fácil

Link: <https://leetcode.com/problems/climbing-stairs/>

Complejidad Algorítmica: Temporal $\mathcal{O}(N)$, Espacial $\mathcal{O}(N)$ (o $\mathcal{O}(1)$ si se usan dos variables).

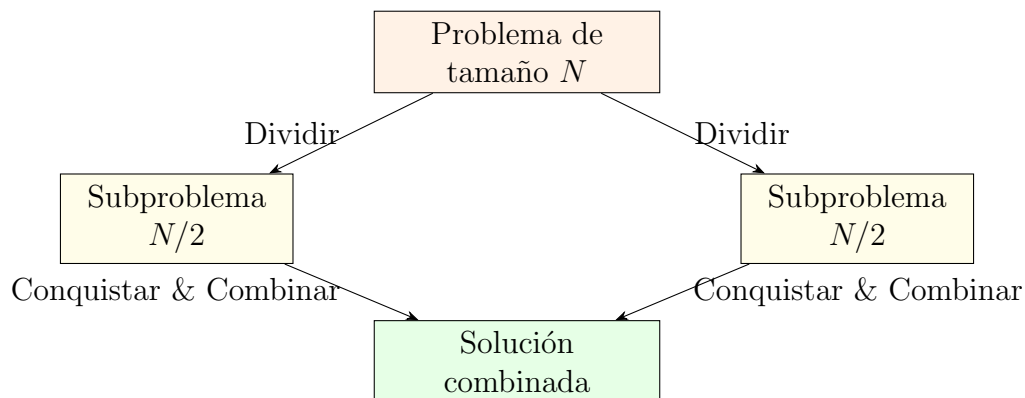
3. Dividir y Conquistar (Divide and Conquer)

3.1. Explicación Técnica

Es un paradigma de diseño de algoritmos que consiste en tres pasos fundamentales:

1. **Dividir** el problema en subproblemas más pequeños e independientes.
2. **Conquistar** (resolver) los subproblemas recursivamente. Si el subproblema es lo suficientemente pequeño, se resuelve directamente (caso base).
3. **Combinar** las soluciones de los subproblemas para formar la solución al problema original grande.

3.2. Diagrama de Componentes del Algoritmo



3.3. Ejemplo y Complejidad

Al aplicar Dividir y Conquistar partiendo a la mitad, usualmente llegamos a algoritmos muy eficientes, calculables por el Teorema Maestro (Master Theorem). **Complejidad Temporal típica:** Suelen ser $\mathcal{O}(N \log N)$ (ej. Merge Sort) o $\mathcal{O}(\log N)$ (ej. Búsqueda Binaria).

3.4. Ejercicio de Práctica en LeetCode

Nombre: Convert Sorted Array to Binary Search Tree

Número: 108

Dificultad: Fácil

Link: <https://leetcode.com/problems/convert-sorted-array-to-binary-search-tree/>

Complejidad Algorítmica: $\mathcal{O}(N)$ temporal, porque visitamos cada nodo (elemento del arreglo) una vez para construir el árbol.

4. Merge Sort

4.1. Explicación Técnica

Merge Sort es un algoritmo de ordenamiento que implementa directamente el paradigma de Dividir y Conquistar. Divide recursivamente el arreglo en dos mitades hasta llegar a arreglos triviales de tamaño 1 (los cuales están ordenados por definición). Luego, combina (*merge*) de forma ordenada las mitades adyacentes hasta reconstruir el arreglo completo.

4.2. Diagrama de Fase de División

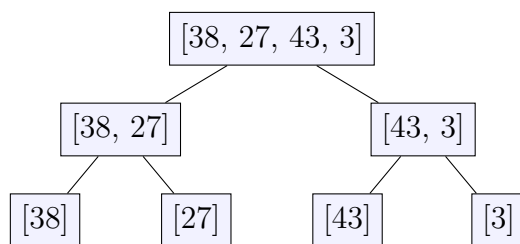


Diagrama: Recursión de división en Merge Sort. El paso inverso realiza la intercalación ordenada (Merge).

4.3. Ejemplo y Complejidad

El proceso de intercalar (*merge*) dos arreglos ordenados toma tiempo lineal. La profundidad del árbol de recursión es logarítmica. **Complejidad Temporal:** $\mathcal{O}(N \log N)$ en todos los casos (peor, mejor y promedio).

Complejidad Espacial: $\mathcal{O}(N)$ porque requiere espacio auxiliar (un arreglo temporal) para hacer la mezcla (*merge*).

4.4. Ejercicio de Práctica en LeetCode

Nombre: Merge Sorted Array

Número: 88

Dificultad: Fácil

Link: <https://leetcode.com/problems/merge-sorted-array/>

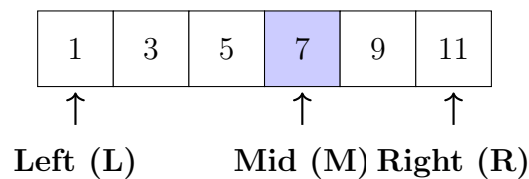
Complejidad Algorítmica: $\mathcal{O}(M + N)$ al mezclar dos arreglos de tamaños M y N . Este es el paso core de la conquista en un Merge Sort.

5. Búsqueda Binaria (Binary Search)

5.1. Explicación Técnica

Es un algoritmo de búsqueda altamente eficiente que encuentra la posición de un valor objetivo dentro de un arreglo **previamente ordenado**. Funciona comparando el valor objetivo con el elemento del medio del arreglo; si no son iguales, elimina la mitad del espacio en la que el objetivo no puede estar y repite la búsqueda en la mitad restante, actualizando punteros.

5.2. Diagrama de Punteros



Búsqueda Binaria evaluando el elemento central (M). Dependiendo del valor, L avanza a M+1 o R retrocede a M-1.

5.3. Ejemplo y Complejidad

Al reducir a la mitad el espacio de búsqueda en cada iteración, el crecimiento es logarítmico.

```
1 int left = 0, right = arr.length - 1;
2 while(left <= right) {
3     int mid = left + (right - left) / 2;
4     if(arr[mid] == target) return mid;
5     if(arr[mid] < target) left = mid + 1; // Descartar izquierda
6     else right = mid - 1; // Descartar derecha
7 }
```

Complejidad Temporal: $\mathcal{O}(\log N)$, muy superior al $\mathcal{O}(N)$ de la búsqueda lineal.

Complejidad Espacial: $\mathcal{O}(1)$ en su versión iterativa.

5.4. Ejercicio de Práctica en LeetCode

Nombre: Binary Search

Número: 704

Dificultad: Fácil

Link: <https://leetcode.com/problems/binary-search/>

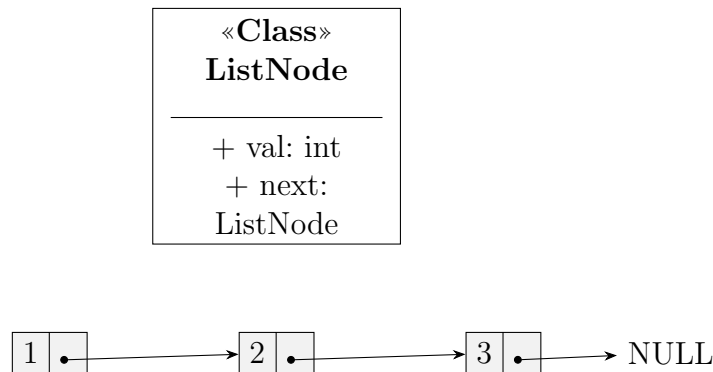
Complejidad Algorítmica: $\mathcal{O}(\log N)$.

6. Listas Enlazadas (Linked Lists)

6.1. Explicación Técnica

Es una estructura de datos lineal compuesta de nodos. A diferencia de un arreglo convencional, los elementos no se almacenan en ubicaciones contiguas de memoria. En una lista enlazada simple, cada nodo contiene dos componentes: el **dato** (valor) y un **puntero** (o referencia) al siguiente nodo en la secuencia. El último nodo apunta a **NULL**.

6.2. Diagrama de Clase y Objetos



Arriba: Diagrama de Clase UML estático. Abajo: Representación dinámica en memoria de los objetos encadenados.

6.3. Ejemplo y Complejidad

Las listas enlazadas destacan por la eficiencia al insertar elementos si ya se tiene la referencia.

- **Insertión / Eliminación al inicio:** $\mathcal{O}(1)$, solo requiere reasignar referencias.
- **Búsqueda (Acceso por índice):** $\mathcal{O}(N)$, se requiere atravesar nodo a nodo desde la cabeza (*head*).

6.4. Ejercicio de Práctica en LeetCode

Nombre: Reverse Linked List

Número: 206

Dificultad: Fácil

Link: <https://leetcode.com/problems/reverse-linked-list/>

Complejidad Algorítmica: Temporal $\mathcal{O}(N)$ al recorrer los nodos una vez; Espacial $\mathcal{O}(1)$ al redirigir los punteros de manera local (*in-place*).